
ClinicAI

Knowledge Transfer Document

FellowAI / ClinicAI — Xccelera AI

Author: Nabil Rizwan

Date: 2026-06-16

Contact: dev@xccelera.ai

ClinicAI — Knowledge Transfer Document

Project: ClinicAI / FellowAI

Author: Nabil Rizwan

Organisation: Xccelera AI

Date: 2026-06-16

Audience: Engineering team taking over the project

Table of Contents

- [Project Overview](#1-project-overview)
 - [Tech Stack](#2-tech-stack)
 - [Repository Layout](#3-repository-layout)
 - [System Architecture](#4-system-architecture)
 - [Data Flow — Inbound WhatsApp Message](#5-data-flow--inbound-whatsapp-message)
 - [LangGraph Agent System](#6-langgraph-agent-system)
 - [Clinical Intelligence Pipeline (Scribe)](#7-clinical-intelligence-pipeline-scribe)
 - [Lab Report Parser Pipeline](#8-lab-report-parser-pipeline)
 - [Persistence Layer](#9-persistence-layer)
 - [Database Schema (PostgreSQL)](#10-database-schema-postgresql)
 - [APScheduler — Background Jobs](#11-apscheduler--background-jobs)
 - [API Reference](#12-api-reference)
 - [Environment Variables](#13-environment-variables)
 - [Deployment](#14-deployment)
 - [Local Development Setup](#15-local-development-setup)
 - [Dashboard (Next.js)](#16-dashboard-nextjs)
 - [Multi-Tenancy Design](#17-multi-tenancy-design)
 - [Known Bugs & Issues](#18-known-bugs--issues)
 - [What Is Built vs. What Is Pending](#19-what-is-built-vs-what-is-pending)
 - [Cost Profile](#20-cost-profile)
 - [How to Extend — Common Tasks](#21-how-to-extend--common-tasks)
 - [Credentials & Secrets Checklist](#22-credentials--secrets-checklist)
-

1. Project Overview

ClinicAI is a **WhatsApp-native clinic management system** for Indian single-doctor clinics. Patients interact entirely over WhatsApp (English or Hinglish) — booking, consulting, uploading lab reports, and receiving follow-ups. Doctors manage their practice via WhatsApp (appointment approvals, voice-note scribing, weekly insights) and a web dashboard.

Core value proposition: Replaces a manual front-desk receptionist and a paper-based clinical records system with a ■350–400/month AI stack (vs. Suki AI at \$299/month).

What it does end-to-end:

- Patient books appointment via WhatsApp → doctor gets WhatsApp approval button → calendar event created
- Doctor sends voice note → Groq Whisper transcribes → LLaMA 70B generates SOAP note → grounding check → SNOMED/RxNorm coding → HL7 FHIR R4 bundle → PDF → doctor approves → PDF sent to patient
- Patient sends lab report PDF → pdfplumber extracts values → Groq summarises and flags critical values → forwarded to doctor
- All clinical data stored as structured `MedicalRecord` rows in PostgreSQL — accessible via dashboard patient timeline

2. Tech Stack

Layer	Technology	Version	Notes
API Framework	FastAPI	0.115	Async, webhook handler
Agent Orchestration	LangGraph	0.3	Multi-agent state machine
Primary LLM	Groq LLaMA 3.3 70B	—	<code>llama-3.3-70b-versatile</code>
Fallback LLM	Google Gemini Flash	2.5	Classifier fallback only
Speech-to-Text	Groq Whisper	large-v3	Doctor voice notes
Messaging	Twilio WhatsApp API	—	All inbound/outbound
Session Store	Redis 7	—	TTL-based state
Database	PostgreSQL 16 + asyncpg	—	Persistent records
Migrations	Alembic	—	3 migrations so far
Scheduler	APScheduler 3.10	—	Reminders, no-show, insights
PDF Generation	ReportLab	—	SOAP note PDFs
PDF Parsing	pdfplumber + pytesseract	—	Lab report extraction
Clinical Standards	SNOMED CT + RxNorm	—	Local JSON + NLM API
Interoperability	HL7 FHIR R4	—	Structured clinical bundles
Auth	python-jose (JWT HS256) + bcrypt	—	Dashboard login
Key Encryption	cryptography (Fernet)	—	Per-clinic API keys at rest
Dashboard	Next.js + Tailwind + shadcn/ui	—	<code>/dashboard</code> directory
Containerization	Docker Compose	—	4 services: api, dashboard, postgres, redis
Reverse proxy / TLS	Caddy	—	Auto HTTPS via sslip.io

3. Repository Layout

```
ClinicAI/
  ■■■ app/
```

- ■■■ api/ # FastAPI routers
- ■ ■■■ webhook_router.py # POST /webhook/twilio – main entry point
- ■ ■■■ auth_router.py # /api/auth/signup, /login, /me
- ■ ■■■ clinic_router.py # /api/clinics CRUD
- ■ ■■■ doctor_api_router.py# /api/clinics/{id}/doctors CRUD
- ■ ■■■ patient_router.py # /api/clinics/{id}/patients + records
- ■ ■■■ appointment_router.py # /api/clinics/{id}/appointments
- ■ ■■■ config_router.py # /api/clinics/{id}/config (AI model config)
- ■ ■■■ scribe_router.py # POST /scribe/consult
- ■ ■■■ parser_router.py # POST /parser/parse-report
- ■ ■■■ classifier_router.py# POST /classify
- ■■■ core/
- ■ ■■■ deps.py # FastAPI dependencies (get_db, get_current_user)
- ■ ■■■ security.py # JWT create/verify, password hash
- ■■■ graph/
- ■ ■■■ classifier.py # Classifier LangGraph – 6-node intent classification
- ■ ■■■ booking.py # Backward-compat shim → router.py
- ■ ■■■ router.py # RouterAgent – top-level orchestrator
- ■ ■■■ agents/
- ■ ■■■ booking_agent.py # Book, cancel, reschedule, status
- ■ ■■■ consultation_agent.py # Active consultation session handling
- ■ ■■■ lab_agent.py # lab_report_share intent (no PDF)
- ■ ■■■ followup_agent.py # followup_query, prescription_request
- ■ ■■■ emergency_agent.py # Emergency → 112 + doctor alert
- ■ ■■■ after_hours_agent.py# Queue message for next open day
- ■ ■■■ parser/
- ■ ■■■ nodes.py # Lab report extraction nodes
- ■ ■■■ pipeline.py # Parser LangGraph assembly
- ■ ■■■ state.py # ParserState schema
- ■ ■■■ scribe/
- ■ ■■■ nodes.py # transcribe, soap_gen, grounding, fhir_coding, pdf_output
- ■ ■■■ pipeline.py # Scribe LangGraph assembly
- ■ ■■■ pdf_builder.py # ReportLab PDF construction
- ■ ■■■ state.py # ScribeState schema
- ■■■ models/ # SQLAlchemy ORM models
- ■ ■■■ clinic.py # Clinic
- ■ ■■■ user.py # ClinicUser (admin/superadmin)
- ■ ■■■ doctor.py # Doctor
- ■ ■■■ patient.py # Patient
- ■ ■■■ appointment.py # Appointment
- ■ ■■■ medical_record.py # MedicalRecord

```

■ ■ ■■■ model_config.py # ModelConfig (per-clinic LLM keys)
■ ■■■ prompts/
■ ■ ■■■ __init__.py # CLASSIFIER_SYSTEM_PROMPT, CLASSIFIER_FEWSHOT,
■ ■ # BOOKING_ENTITY_PROMPT (all LLM prompts except
■ ■ # scribe/nodes.py inline prompts)
■ ■■■ schemas/
■ ■ ■■■ __init__.py # Pydantic models: BookingSession, BookingState,
■ ■ # AppointmentRecord, ConsultationSession, ClassifierState
■ ■■■ services/
■ ■ ■■■ store.py # Redis + in-memory fallback – all state management
■ ■ ■■■ whatsapp.py # Twilio send/download helpers
■ ■ ■■■ scheduler.py # APScheduler job definitions
■ ■ ■■■ appointment_approval.py # Doctor approval flow
■ ■ ■■■ clinical_scribe.py # Voice-note → scribe pipeline trigger
■ ■ ■■■ consultation_service.py # Consultation finalize-and-send
■ ■ ■■■ doctor.py # Doctor WhatsApp command handler
■ ■ ■■■ identity.py # Doctor number registry
■ ■ ■■■ doctor_directory.py # Doctor name matching / clinic directory
■ ■ ■■■ google_calendar.py # Calendar event creation + slot resolution
■ ■ ■■■ llm_factory.py # get_llm_for_vendor() – per-clinic LLM instantiation
■ ■ ■■■ patient_service.py # upsert_patient, MedicalRecord write-through
■ ■ ■■■ pdf_service.py # Lab PDF handling
■ ■ ■■■ terminology.py # SNOMED CT + RxNorm lookup (local → NLM API)
■ ■ ■■■ async_runner.py # run_async() for sync→async bridging
■ ■■■ database.py # SQLAlchemy async engine + Base
■■■■ dashboard/ # Next.js frontend (separate npm project)
■■■■ alembic/ # Database migrations
■ ■■■ versions/
■ ■■■ 001_add_google_calendar_id_to_doctors.py
■ ■■■ 002_add_doctor_name_to_medical_records.py
■ ■■■ 003_add_appointments_table.py
■■■■ .env.example # All env vars with descriptions
■■■■ Dockerfile # API container
■■■■ docker-compose.yml # Local dev (direct ports, no TLS)
■■■■ docker-compose.prod.yml # Production (Caddy TLS)
■■■■ Caddyfile # Caddy HTTPS config
■■■■ DEPLOY.md # Step-by-step VPS deployment guide
■■■■ SETUP_FOR_TESTING.md # Manual testing walkthrough
■■■■ ENGINEERING_REPORT.md # Detailed technical report
■■■■ WHATSAPP_MIGRATION.md # Twilio WhatsApp number migration guide

```

[illegible]

6 nodes, compiled once at import time as `classifier_graph` :

```
START → validate_node → preprocess_node → classify_node
↓ (on error)
fallback_node → postprocess_node → END
↓ (on success)
postprocess_node → END
validate_node (invalid) → error_node → END
```

10 valid intents:

`appointment_book` , `appointment_cancel` , `appointment_reschedule` , `appointment_status` , `followup_query` , `lab_report_share` , `prescription_request` , `general_query` , `emergency` , `consultation_message`

Output shape from classifier:

```
{
  "intent": str,
  "confidence": float, # 0.0-1.0
  "entities": {
    "patient_name": str|None,
    "doctor_name": str|None,
    "requested_date": str|None,
    "requested_time": str|None,
    "symptoms_mentioned": str|None,
    "medication_mentioned": str|None,
  },
  "bot_response": str|None, # pre-generated reply from LLM
  "all_intents": list[dict], # multi-intent support
  "is_multi_intent": bool,
  "is_injection": bool,
  "is_emergency": bool,
}
```

Per-clinic LLM config is forwarded via `llm_vendor` , `llm_model` , `llm_enc_key` state fields. The `get_llm_for_vendor()` factory (`app/services/llm_factory.py`) decrypts the Fernet-encrypted key and builds the LangChain LLM object.

6.2 Router Graph (`app/graph/router.py`)

Top-level orchestrator — compiled as `router_graph` with Redis (or MemorySaver fallback) checkpointer.

Routing logic in `route_after_session()` :

- `emergency` intent → always → `emergency_dispatch_node`
- `CONSULTATION_ACTIVE` journey state OR `consultation_message` (not from `NEW/BOOKING/POST_CONSULT/FOLLOW_UP`) → `consultation_dispatch_node`
- Same ^ but clinic is closed → `after_hours_dispatch_node`

- `lab_report_share` → `lab_dispatch_node`
- `followup_query` / `prescription_request` → `followup_dispatch_node`
- Explicit appointment management intents → `booking_dispatch_node` (even from `POST_CONSULT`)
- `POST_CONSULT` / `FOLLOW_UP_PENDING` with other message → `followup_dispatch_node`
- Everything else → `booking_dispatch_node`

6.3 Booking Agent (`app/graph/agents/booking_agent.py`)

Handles: `appointment_book` , `appointment_cancel` , `appointment_reschedule` , `appointment_status` , `general_query` , off-topic messages.

Booking session states (`BookingSession.state`):

```

GREETING → COLLECTING_INFO → COLLECT_DOCTOR_PREFERENCE → CONFIRM_SLOT
→ WAITING_DOCTOR_APPROVAL → BOOKED
→ RESCHEDULE_COLLECTING → RESCHEDULE_CONFIRM
→ CANCEL_CONFIRM
→ SELECT_APPOINTMENT_CANCEL | SELECT_APPOINTMENT_RESCHEDULE

```

Journey states (`BookingSession.journey_state`):

```

NEW_PATIENT → BOOKING_IN_PROGRESS → AWAITING_CONFIRMATION → BOOKED
→ CONSULTATION_ACTIVE → POST_CONSULT → FOLLOW_UP_PENDING

```

Key nodes: `flow_node` (main booking flow), `confirm_node` , `cancel_node` , `reschedule_node` , `appointment_status_node` , `off_topic_node` .

Entity extraction uses a separate LLM call (`BOOKING_ENTITY_PROMPT`) to parse date/time/name from free-text. The classifier also extracts entities but the booking agent re-extracts for mid-flow messages.

6.4 Consultation Agent (`app/graph/agents/consultation_agent.py`)

- Detects if a patient is in an active consultation
- Labels sender role (doctor/patient) based on `identity.role`
- Appends messages atomically to `ConsultationSession` in Redis (Lua script prevents race conditions)
- Resets the 30-minute inactivity timeout on each message (`schedule_consultation_timeout`)
- Detects closing phrases (`ok` , `done` , `take care` , `bye`) → `finalize_and_send()`
- `finalize_and_send()` → runs scribe pipeline → writes `MedicalRecord` to PostgreSQL → schedules follow-up message

6.5 Emergency Agent (`app/graph/agents/emergency_agent.py`)

- Immediately replies with 112 emergency instructions
- Sends WhatsApp alert to all configured doctor numbers
- Resets session to `GREETING`

6.6 After-Hours Agent (`app/graph/agents/after_hours_agent.py`)

- Queues the message in Redis (`clinicai:afterhours:{doctor_number}`) with TTL 36h
- Sends patient acknowledgment: "clinic is closed, we'll follow up at opening"
- APScheduler flushes the queue daily at `CLINIC_OPEN_HOUR`

7. Clinical Intelligence Pipeline (Scribe)

Trigger paths:

- **Voice note during active consultation** → buffered into ConsultationSession → finalized on close
- **Voice note sent by doctor outside consultation** → `handle_doctor_voice_note()` → scribe pipeline directly
- **POST /scribe/consult** → external consultation bundle → scribe pipeline

Pipeline nodes (`app/graph/scribe/nodes.py`):

```
transcribe_node (Groq Whisper → transcript + language)
↓
soap_generator_node (LLaMA 70B → structured SOAP JSON with per-section confidence)
↓
grounding_check_node (LLaMA 70B → verifies every SOAP sentence maps to transcript)
↓
extract_entities_node (LLaMA 70B → {symptoms, medications, diagnoses})
↓
fhir_coding_node (local JSON table → NLM API fallback → SNOMED CT + RxNorm codes
→ HL7 FHIR R4 Bundle)
↓
followup_generator_node (LLaMA 70B → 2-3 follow-up questions + WhatsApp summary ≤300 chars)
↓
pdf_output_node (ReportLab → SOAP note PDF with SNOMED codes + FHIR reference)
```

SOAP note JSON schema:

```
{
  "patient_name": "",
  "doctor_name": "",
  "date": "",
  "follow_up_days": null,
  "subjective": { "content": "", "confidence": 0.0, "is_missing": false, "clarifying_question": "" },
  "objective": { "content": "", "confidence": 0.0, "is_missing": false, "clarifying_question": "" },
  "assessment": { "content": "", "confidence": 0.0, "is_missing": false, "clarifying_question": "" },
  "plan": { "content": "", "confidence": 0.0, "is_missing": false, "clarifying_question": "" }
}
```

Confidence threshold: 0.6. Sections below 0.6 are flagged to the doctor as needing review.

Scribe approval flow:

- SOAP PDF generated → `save_pending_soap()` in Redis → doctor receives PDF link + Approve/Reject buttons (Twilio Content Template `SOAP_APPROVAL_CONTENT_SID`)
- Doctor replies APPROVE `RXxxxxx` or REJECT `RXxxxxx` (or taps button)
- `handle_doctor_message()` in `app/services/doctor.py` parses the command → on approve: sends PDF to patient, writes `MedicalRecord` to PostgreSQL, schedules follow-up

Key scribe service: `app/services/clinical_scribe.py` — wires voice-note download → temp file → scribe pipeline → approval flow.

8. Lab Report Parser Pipeline

Trigger: Patient sends PDF as WhatsApp attachment → `handle_incoming_pdf()` in `app/services/pdf_service.py`

Pipeline (`app/graph/parser/`):

- pdfplumber text extraction (OCR fallback via pytesseract if text layer is empty)
- LLM panel detection (`CBC` , `LFT` , `KFT` , `Lipid` , `Thyroid`)
- Value extraction with reference ranges
- Critical value flagging
- Groq summary generation
- PDF report generation (ReportLab)
- Summary + PDF forwarded to doctor via WhatsApp

Lab reports are stored as `MedicalRecord` with `record_type="lab_report"` containing `lab_results = {all_values, abnormals, criticals}` .

9. Persistence Layer

Redis (`app/services/store.py`)

Redis is primary; in-memory Python dicts are the fallback (with a one-time warning at startup). All public functions have identical signatures regardless of backend.

Key schema (all prefixed `clinica:`):

Key pattern	Data	TTL
<code>session:{clinic_id}:{phone}</code>	BookingSession JSON	24h
<code>session:{phone}</code>	BookingSession JSON (legacy)	24h
<code>appt:{id}</code>	AppointmentRecord JSON	no TTL
<code>appts_by_phone:{phone}</code>	Redis SET of appointment_ids	no TTL
<code>approval:{id}</code>	Approval dict	no TTL
<code>approvals_waiting:{doctor}</code>	Redis SET of approval_ids	no TTL
<code>doctor_profile:{phone}</code>	Doctor profile dict	no TTL
<code>greeted:{phone}</code>	"1"	30 days
<code>slot_suggestions:{phone}</code>	JSON list	24h
<code>soap:{id}</code>	Pending SOAP dict	7 days
<code>lab:{id}</code>	Pending lab review dict	7 days
<code>last_active:{phone}</code>	ISO timestamp	7 days
<code>consult:{clinic_id}:{phone}</code>	ConsultationSession JSON	4h

Key pattern	Data	TTL
afterhours:{doctor_number}	JSON list of queued messages	36h
doctor_reply_ctx:{doctor}	JSON list (up to 10 patients)	2h
pdf:{namespace}:{doc_id}	{"path": ...}	7 days

Atomic consultation-message append: Uses a Lua script (`_APPEND_MSG_LUA`) registered on the Redis connection. Falls back to per-patient `threading.Lock` for the in-memory path. This prevents lost-update races when multiple WebSocket messages arrive for the same patient.

PostgreSQL

Async SQLAlchemy via `asyncpg`. Engine in `app/database.py` . All models in `app/models/` . Tables auto-created on startup via `Base.metadata.create_all()` .

See Section 10 for full schema.

10. Database Schema (PostgreSQL)

clinics

`id` (UUID PK), `name` , `twilio_number` (unique), `timezone` , `open_hour` , `close_hour` , `is_active` , `created_at`

clinic_users

`id` (UUID PK), `email` (unique), `hashed_password` (bcrypt), `full_name` , `role` (`admin` / `superadmin`), `clinic_id` (FK), `is_active` , `created_at`

doctors

`id` (UUID PK), `clinic_id` (FK), `name` , `specialty` , `whatsapp_number` , `working_hours_start` , `working_hours_end` , `appointment_duration_minutes` (default 30), `buffer_minutes` (default 5), `is_active` , `created_at`

model_configs

`id` (UUID PK), `clinic_id` (FK unique), `llm_vendor` , `llm_model` , `stt_model` , `groq_api_key_enc` , `anthropic_api_key_enc` , `openai_api_key_enc` , `google_api_key_enc` (all Fernet-encrypted), `updated_at`

patients

`id` (UUID PK), `clinic_id` (FK), `phone_number` (indexed), `name` , `age` , `gender` , `blood_group` , `allergies` (JSON), `chronic_conditions` (JSON), `current_medications` (JSON), `doctor_notes` , `is_active` , `created_at` , `last_visit_at`

appointments

`id` (PK, APTxxxxx format matching Redis), `clinic_id` , `patient_id` , `doctor_id` , `from_number` , `patient_name` , `doctor_name` , `date_str` , `time_str` , `appointment_datetime` , `symptoms`

(JSON), status (active / cancelled / completed), confirmed_at , reminder_sent , created_at , updated_at

Unique constraint: (clinic_id, from_number, doctor_name, appointment_datetime) — prevents duplicate bookings.

medical_records

id (UUID PK), patient_id (FK), clinic_id (FK), doctor_id (FK nullable), visit_date , record_type (consultation / lab_report / booking), chief_complaint , soap_subjective/objective/assessment/plan (Text), soap_confidence (Float), diagnoses (JSON), medications (JSON), symptoms (JSON), lab_panel_type , lab_results (JSON), fhir_bundle (JSON), pdf_url , created_at

11. APScheduler — Background Jobs

Scheduler defined in `app/services/scheduler.py` . Uses SQLAlchemy job store (PostgreSQL > SQLite fallback) for persistence across restarts.

Important: In multi-worker deployments, set `DISABLE_SCHEDULER=true` on all workers except one to prevent duplicate jobs.

Job	Trigger	What it does
reminder_{appt_id}	Date: appt_time - REMINDER_MINUTES_BEFORE (default 2h)	WhatsApp reminder to patient
noshow_1hr_{appt_id}	Date: appt_time + 1h	No-show recovery message (attempt 1)
noshow_24hr_{appt_id}	Date: appt_time + 24h	No-show recovery message (attempt 2)
consult_timeout_{patient}	Date: now + CONSULTATION_TIMEOUT_MINUTES (default 30)	Finalize consultation on inactivity
afterhours_flush_{doctor}	Cron: daily at CLINIC_OPEN_HOUR:00	Re-inject queued after-hours messages
followup_{patient}	Date: now + follow_up_days (from SOAP or default 2)	Send follow-up check-in to patient
weekly_insights_{doctor}	Cron: every Monday at WEEKLY_INSIGHTS_HOUR:00 (default 8)	7-day practice summary to doctor

Demo mode overrides:

- `DEMO_REMINDER_DELAY_MINUTES` → fires reminder N minutes after approval (ignores appointment time)
- `DEMO_FOLLOWUP_DELAY_MINUTES` → fires follow-up N minutes after consultation
- `CONSULTATION_TIMEOUT_MINUTES=2` → 2-minute timeout for demo

12. API Reference

Webhook (main entry point)

Method	Path	Description
POST	/webhook/twilio	All inbound WhatsApp traffic (patients + doctors)
GET	/webhook/twilio	Health check

Auth

Method	Path	Description
POST	/api/auth/signup	Create clinic + admin user + ModelConfig; returns JWT
POST	/api/auth/login	Email/password → JWT
GET	/api/auth/me	Current user profile

Clinics

Method	Path	Notes
GET	/api/clinics/	Superadmin only
GET/PUT	/api/clinics/{id}	Admin or superadmin
DELETE	/api/clinics/{id}	Superadmin (soft-delete)

Doctors / Patients / Appointments / Config

All nested under `/api/clinics/{clinic_id}/` :

- `doctors/` — CRUD
- `patients/` — list, get by phone, detail, update profile, records timeline
- `appointments/` — list with filters, update status (updates both PG + Redis)
- `config/` — LLM config, test connection

Clinical

Method	Path	Description
POST	/classify	Direct classifier test
POST	/scribe/consult	Consultation bundle → SOAP result
GET	/scribe/pdf/{id}	Serve SOAP note PDF
POST	/parser/parse-report	Lab report extraction
GET	/lab-report/pdf/{id}	Serve lab report PDF

Debug (requires `DEBUG_ENDPOINTS=true` + auth)

`/debug/sessions` , `/debug/appointments` , `/debug/identity` , `/debug/pending-approvals` ,
`/debug/doctors` , `/debug/consultations`

Health

GET / → {"status":"ok", ...} with feature flags

13. Environment Variables

Variable	Required	Default	Description
GROQ_API_KEY	Yes	—	Groq (LLaMA + Whisper)
GROQ_MODEL	No	llama-3.3-70b-versatile	Primary LLM model
WHISPER_MODEL	No	whisper-large-v3	STT model
GEMINI_API_KEY	Recommended	—	Gemini fallback for classifier
GEMINI_MODEL	No	gemini-2.5-flash	
TWILIO_ACCOUNT_SID	Yes	—	
TWILIO_AUTH_TOKEN	Yes	—	Used for webhook signature validation
TWILIO_WHATSAPP_FROM	Yes	—	e.g. whatsapp:+14155238886
SOAP_APPROVAL_CONTENT_SID	No	—	Twilio template SID for SOAP buttons
APPOINTMENT_APPROVAL_CONTENT_SID	No	—	Twilio template SID for booking buttons
DOCTOR_WHATSAPP_NUMBERS	Yes	—	CSV: Dr Name:+91XXXXXXXXXX, Dr Name2:+91...
DATABASE_URL	Yes	—	postgresql+asyncpg://user:pass@host:5432/db
REDIS_URL	Yes	—	redis://localhost:6379
SECRET_KEY	Yes	—	JWT signing key (≥32 chars)
ENCRYPTION_KEY	Yes	—	Fernet base64 key (generate: see below)
PUBLIC_BASE_URL	Yes	—	Public URL of API (for PDF links)
DASHBOARD_URL	No	http://localhost:3000	CORS allow origin
CLINIC_NAME	No	ClinicAI	Display name
CLINIC_OPEN_HOUR	No	9	24h format
CLINIC_CLOSE_HOUR	No	20	24h format
REMINDER_MINUTES_BEFORE	No	120	2h before appointment
DEMO_REMINDER_DELAY_MINUTES	No	—	Set 2 for demo
CONSULTATION_TIMEOUT_MINUTES	No	30	Set 2 for demo
FOLLOWUP_DEFAULT_DAYS	No	2	

Variable	Required	Default	Description
DEMO_FOLLOWUP_DELAY_MINUTES	No	—	Set 3 for demo
GOOGLE_CALENDAR_ENABLED	No	False	
GOOGLE_CALENDAR_CREDENTIALS_FILE	No	google_credentials.json	
GOOGLE_CALENDAR_TOKEN_FILE	No	google_token.json	
GOOGLE_CALENDAR_ID	No	primary	
GOOGLE_CALENDAR_TIMEZONE	No	Asia/Kolkata	
APPOINTMENT_DURATION_MINUTES	No	30	
DISABLE_SCHEDULER	No	—	Set true on extra workers
DEBUG_ENDPOINTS	No	—	Set true to expose debug routes
SEED_DEMO_DOCTORS	No	false	Seed demo data on startup
JAMEEL_SCRIBE_URL	No	—	Remote scribe API; empty = run locally
WEBHOOK_PUBLIC_URL	No	—	Override URL for Twilio signature validation

Generate Fernet key:

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

14. Deployment

Full instructions are in `DEPLOY.md`. Summary:

Prerequisites

- Ubuntu 24.04 VPS (Hetzner CX22 ~€4/mo or similar)
- Docker + Compose plugin installed

Steps

```
# 1. On the server
git clone <repo> clinicaai && cd clinicaai

# 2. Configure environment
cp .env.example .env
nano .env # fill all values, set API_HOST/APP_HOST to <IP>.sslip.io

# 3. Upload Google OAuth files (gitignored)
scp google_credentials.json google_token.json user@<IP>:~/clinicaai/
```

```
# 4. Launch
docker compose -f docker-compose.prod.yml up -d --build

# 5. Set Twilio webhook
# In Twilio Console → WhatsApp sandbox → inbound URL:
# https://api.<IP>.sslip.io/webhook/twilio
```

Services started by docker-compose.prod.yml

- `postgres` — PostgreSQL 16 on port 5432 (internal)
- `redis` — Redis 7 on port 6379 (internal)
- `api` — FastAPI + uvicorn on port 8000 (behind Caddy)
- `dashboard` — Next.js on port 3000 (behind Caddy)
- `caddy` — reverse proxy, auto HTTPS via Let's Encrypt

Verify deployment

```
curl https://api.<IP>.sslip.io/health # → {"status":"ok"}
```

Update after code change

```
git pull
docker compose -f docker-compose.prod.yml up -d --build
```

Database migrations

New deploys: `create_all` handles schema automatically.

Existing DB upgrade:

```
docker compose -f docker-compose.prod.yml exec api alembic upgrade head
```

15. Local Development Setup

Option A — Full Docker

```
cp .env.example .env # edit with your API keys
docker compose up --build
```

- API: `http://localhost:8000/docs`
- Dashboard: `http://localhost:3000`

Option B — Native (recommended for development)

```
# 1. Data services only
docker compose up postgres redis -d

# 2. Backend
python -m venv .venv
.venv\Scripts\activate # Windows
```

```
source .venv/bin/activate # macOS/Linux

pip install -r requirements.txt

cp .env.example .env # fill in keys, set DATABASE_URL to localhost

# 3. Start with ngrok tunnel for Twilio
python dev_start.py # auto-starts ngrok, writes PUBLIC_BASE_URL, starts uvicorn

# 4. Dashboard (separate terminal)
cd dashboard
npm install
npm run dev
```

`dev_start.py` prints the Twilio webhook URL — paste it in the Twilio console.

16. Dashboard (Next.js)

Located in `dashboard/` . Separate npm project.

Key files:

- `dashboard/.env.local` — `NEXT_PUBLIC_API_URL` (API base URL)
- Authentication: JWT stored in `localStorage` , sent as `Authorization: Bearer <token>` header
- Pages: see README "Dashboard Routes" section for full list

Main routes:

- `/auth/signup` → POST `/api/auth/signup` → JWT → redirect `/onboarding`
- `/onboarding/*` → 5-step clinic setup wizard
- `/dashboard` → clinic overview, doctor cards, WhatsApp link
- `/dashboard/patients/[id]` → full medical history timeline (SOAP + lab records with SNOMED/RxNorm chips)
- `/dashboard/appointments` → full appointment table with status filters
- `/dashboard/config` → AI model selection + API key entry + live connection test
- `/admin` → superadmin clinic list

17. Multi-Tenancy Design

The system is multi-tenant from the ground up:

- **Clinic identification:** Every inbound webhook resolves the clinic by matching `To` (Twilio number) → `Clinic.twilio_number` in PostgreSQL.
- **Data isolation:** All DB tables have `clinic_id` FK. All Redis keys use `clinicai:session:{clinic_id}:{phone}` (legacy unscoped key maintained for background jobs).
- **Per-clinic LLM:** `ModelConfig` stores Fernet-encrypted API keys per clinic. The `get_llm_for_vendor()` factory decrypts and instantiates the appropriate LangChain LLM for each request.
- **Per-clinic hours:** `Clinic.open_hour` and `Clinic.close_hour` flow through the entire graph via state fields.

- **Doctor registry:** Each doctor's WhatsApp number is stored in PostgreSQL per clinic. The `identity.py` service also supports `DOCTOR_WHATSAPP_NUMBERS` env var for simple single-clinic setups.

18. Known Bugs & Issues

These were identified in a full codebase audit on 2026-05-19. Status as of 2026-06-16:

Critical Bugs

#	File	Issue	Fix
1	<code>scheduler.py:188</code>	FIXED — reminder fires <code>appt_time - REMINDER_MINUTES</code> ✓	Was now <code>+ 5min</code> originally
2	<code>router.py:session_node</code>	FIXED — session loaded from Redis before booking graph ✓	Was not loaded before
3	<code>parser/nodes.py:~116</code>	Lab report parser truncates at 6000 chars — only first page	Increase limit or add page-by-page processing
4	<code>appointment_approval.py</code>	Google Calendar errors silently block booking	Any GCal exception returns False (slot unavailable)
5	<code>appointment_approval.py:~300</code>	Slot candidates ignore doctor working hours	Use <code>Doctor.working_hours_start/end</code> from DB

Security Issues

#	File	Issue
6	<code>main.py</code> (startup)	CORS <code>allow_origins=["*"]</code> — tighten to <code>DASHBOARD_URL</code> in production
7	<code>webhook_router.py:417-438</code>	PDF download endpoints (<code>/lab-report/pdf/{id}</code> , <code>/scribe/pdf/{id}</code>) serve files from temp paths — validate <code>document_id</code> is alphanumeric only
8	<code>clinical_scribe.py:~173</code>	Weak patient matching — substring match on name for SOAP PDF delivery

Hardcoding

#	File	Issue
9	Multiple nodes	LLM model names should all read from env (most already do; verify <code>parser/nodes.py</code>)
10	<code>classifier.py:27</code>	<code>GEMINI_MODEL</code> env fallback works ✓
11	<code>scribe/nodes.py:113</code>	<code>WHISPER_MODEL</code> env fallback works ✓
12	<code>booking_agent.py:49-160</code>	<code>MSG_*</code> templates are hardcoded strings (not bad per se, but note if you want i18n)

#	File	Issue
13	<code>.env.example</code>	DEFAULT_APPOINTMENT_YEAR=2026 — should not be hardcoded; derive from current date

Architecture Limitations

#	Issue
L1	APScheduler in-memory job store: restart loses all pending jobs (reminders, no-show checks). FIXED — now uses SQLAlchemy job store (PostgreSQL > SQLite fallback)
L2	LangGraph MemorySaver: booking checkpoint lost on restart. FIXED — now uses RedisSaver with MemorySaver fallback
L3	MedicalRecord has no appointment_id FK — consultation records can't be linked back to the exact appointment
L4	No test suite — all verification is manual (see <code>SETUP_FOR_TESTING.md</code>)
L5	Doctor cannot send closing phrase to finalize consultation — must come from patient side or timeout
L6	No CI/CD pipeline

19. What Is Built vs. What Is Pending

Based on the FellowAI Sprint Plan (May 23 – Jul 17, 2026):

Completed (Sprint 1 + Sprint 2)

- Context-aware intent classifier (10 intents, Hinglish, Gemini fallback)
- Redis session state machine (7 journey states)
- Full booking multi-turn dialogue (book, cancel, reschedule, status)
- APScheduler reminders (2 hours before slot)
- Cancel + reschedule flows (multi-appointment selection)
- No-show detection (+1hr, +24hr recovery messages)
- LangGraph multi-agent: Router → {Booking, Consultation, Lab, Emergency, AfterHours, Followup}
- ConsultationAgent: message buffering, sender-role labeling, inactivity timeout
- Consultation finalization → scribe pipeline → SOAP → FHIR → PDF → MedicalRecord
- After-hours queue + morning flush
- Emergency handler (112 alert + doctor notify)
- Post-consult follow-up agent
- Clinical scribe (Whisper → SOAP → grounding → entity extraction → SNOMED/RxNorm → FHIR → PDF)
- Lab report parser (5 panel types: CBC, LFT, KFT, Lipid, Thyroid)
- Follow-up question generator
- Weekly practice insights (every Monday)
- Doctor WhatsApp commands (`today` , `pending` , appointment approve/reject, SOAP approve/reject)
- Doctor onboarding setup via WhatsApp
- PostgreSQL patient profiles (name, phone, last visit, medical records timeline)
- Multi-tenant with per-clinic LLM config (Groq/Anthropic/OpenAI/Google)

- Next.js dashboard (patients, appointments, doctors, AI config)
- JWT auth + bcrypt
- Per-clinic API key encryption (Fernet)
- Docker Compose with Caddy TLS
- Doctor registration from DB (not just env var)

Pending (Sprint 3 + Handover, as of 2026-06-16)

- **Stress testing** — 20 concurrent patient sessions
- **Input sanitization** — emojis, forwarded messages, image attachments, stickers
- **Doctor WhatsApp commands** — cancel slot 3pm, mark Rahul as seen (partial — today / pending works)
- **Rate limiting** — throttle at >10 msgs/60s per patient
- **Patient profile enrichment** — age, gender, allergies, chronic conditions via WhatsApp conversation
- **MedicalRecord** ↔ **appointment FK** — linking each consultation to the specific appointment
- **Test suite** — automated unit and integration tests
- **CI/CD pipeline**
- Suggest Time **button** on appointment approval (feedback from Demo 0)
- **SNOMED/HL7 validation** — improve coverage; currently uses local JSON + NLM API fallback
- what we would build next **document** (internship handover deliverable)
- **DPDP Act 2023 compliance audit** for production use with real patient data

20. Cost Profile

Demo scale: 1 clinic, 600 appointments/month, 400 voice notes, 50 lab reports

Service	Cost
Groq Whisper (400 voice notes)	~\$0.40/mo
Groq LLaMA 70B (all LLM calls)	~\$0.80/mo
Twilio WhatsApp	~\$345/mo
Hosting (Hetzner CX22)	~\$4/mo
Total	~\$350–400/month (~\$5–7)

10-clinic scale: ~\$3,500–4,000/month (~\$50–60)

vs. Suki AI: \$299/month/doctor — ClinicAI is ~60× cheaper at demo scale.

LLM recommendation: Keep Groq API. Self-hosting a 7B clinical model costs \$300–600/month GPU vs \$5/month Groq — not worth it at this scale.

21. How to Extend — Common Tasks

Add a new intent

- Add the intent string to `VALID_INTENTS` in `app/graph/classifier.py:38`

-
- Update `CLASSIFIER_SYSTEM_PROMPT` in `app/prompts/__init__.py` to describe the new intent with examples
 - Add routing logic in `route_after_session()` in `app/graph/router.py`
 - Create or extend the target agent to handle the intent

Add a new doctor command

In `app/services/doctor.py`, extend the `handle_doctor_message()` function. Commands are matched via simple string checks on `message_text`.

Add a new LLM vendor

In `app/services/llm_factory.py`, add a case to `get_llm_for_vendor()`. Update `ModelConfig` (model + migration) to store the new vendor's encrypted key. Update `_build_llm_state_fields()` in `webhook_router.py`.

Connect a new WhatsApp number (new clinic)

- In Twilio Console → register new number
- Sign up via dashboard `/auth/signup` → complete onboarding wizard (sets Twilio number, doctors, LLM config)
- Set Twilio inbound webhook to `https://<your-api>/webhook/twilio`
- The system auto-routes by matching `To` field to `Clinic.twilio_number`

For full guide see `WHATSAPP_MIGRATION.md`.

Modify the SOAP note

The full prompt is in `SOAP_SYSTEM` string constant in `app/graph/scribe/nodes.py:152`. The PDF layout is in `app/graph/scribe/pdf_builder.py`.

Add a new scheduler job

In `app/services/scheduler.py`, define a `_job_function()` and a `schedule_*` function that calls `scheduler.add_job()`. Call the `schedule_*` function from the appropriate trigger point (usually `appointment_approval.py` or `consultation_service.py`).

22. Credentials & Secrets Checklist

Before going to production, verify and rotate:

- `[ ] GROQ_API_KEY` — `https://console.groq.com/keys`
- `[ ] GEMINI_API_KEY` — `https://aistudio.google.com/apikey`
- `[ ] TWILIO_ACCOUNT_SID + TWILIO_AUTH_TOKEN` — Twilio Console
- `[ ] TWILIO_WHATSAPP_FROM` — your Twilio WhatsApp number
- `[ ] SOAP_APPROVAL_CONTENT_SID` — Twilio Content Templates
- `[ ] APPOINTMENT_APPROVAL_CONTENT_SID` — Twilio Content Templates
- `[ ] SECRET_KEY` — fresh ≥32 char random string
- `[ ] ENCRYPTION_KEY` — fresh Fernet key (see generation command in Section 13)
- `[ ] POSTGRES_PASSWORD` — strong random password

-
- [] DATABASE_URL — using the same password
 - [] DOCTOR_WHATSAPP_NUMBERS — correct production doctor numbers
 - [] google_credentials.json + google_token.json — uploaded separately (gitignored)
 - [] Twilio webhook URL updated to production API URL
 - [] CORS tightened from * to DASHBOARD_URL
 - [] DEBUG_ENDPOINTS NOT set to true in production
 - [] All patient data during development is synthetic (DPDP Act 2023 requirement)
-

For questions about the codebase, refer to ENGINEERING_REPORT.md for a deep-dive technical breakdown, or SETUP_FOR_TESTING.md for the manual end-to-end test guide.